

Wollo University
Department of Health Informatics
Internet Programming I

Instructor Name :- Daniel Bekele

Chapter 4: Introduction to Hyper Text Markup Language (HTML)

4.1 Introduction to Markup Languages

HTML, or Hypertext Markup Language, is the standard language used to create and structure content on the web. It uses tags to define how text, images, and other multimedia elements are organized and displayed by web browsers, forming the basic structure of a webpage.

- **Markup language:** HTML is a markup language, not a programming language, meaning it uses tags to annotate text and define its structure, such as headings, paragraphs, and links.
- **Role on a webpage:** It provides the structural framework for a webpage, while other technologies like CSS handle styling and JavaScript adds interactivity.
- **"Hypertext":** The "hypertext" part refers to the links that connect web pages to one another, allowing users to navigate between different documents

Example :-

Code

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>This is a Heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

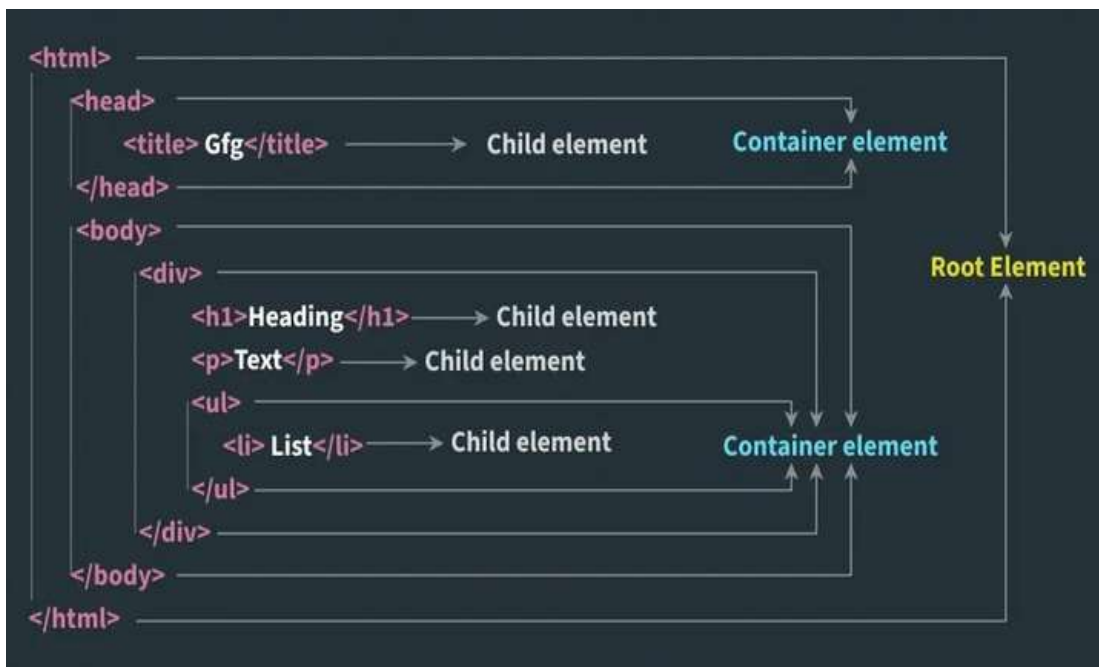
4.2 Document Object Model (DOM) of HTML

The Document Object Model (DOM) of HTML is a programming interface for web documents. It represents the structure of an HTML or XML document as a logical tree, where each node in the tree represents a part of the document, such as an element, attribute, or text content. The DOM allows programming languages, primarily JavaScript, to interact with and manipulate the content, structure, and style of a web page after it has been loaded in the browser.

Key aspects of the HTML DOM:

- **Tree Structure:** The DOM organizes the document as a hierarchical tree of nodes. The `<html>` element is typically the root node, with `<head>` and `<body>` as its children, and so on.
- **Nodes:** Each part of the HTML document is represented as a node in the DOM tree. Common node types include:
 - **Element Nodes:** Represent HTML elements (e.g., `<div>`, `<p>`, `<h1>`).
 - **Text Nodes:** Represent the text content within elements.
 - **Attribute Nodes:** Represent attributes of HTML elements (e.g., `id`, `bgcolor`, `src`).
 - **Document Node:** The top-level node representing the entire document

It acts as a structured representation of an HTML document, allowing developers to programmatically access, modify, and control the content and structure of a web page.



The HTML Document Object Model (DOM) is a tree structure, where each HTML tag becomes a node in the hierarchy. At the top, the `<html>` tag is the root element, containing both `<head>` and `<body>` as child elements.

These in turn have their own children, such as `<title>`, `<div>`, and nested elements like `<h1>`, `<p>`, `<ul>`, and `<li>`. Elements that contain other elements are labeled as container elements, while elements that do not are simply child elements.

This hierarchy allows developers to navigate and manipulate web page content using JavaScript by traversing from parent to child and vice versa.

4.3 Headings, Paragraph and Breaks

Headings, paragraphs, and line breaks are fundamental text formatting elements used to structure and organize content.

Headings (like `<h1>` to `<h6>`) define titles and subtitles, creating a hierarchy to improve scannability.

Paragraphs (`<p>`) group related sentences, with browsers typically adding a gap between them.

A line break (`<br>`) forces content onto a new line without the extra space of a paragraph break.

Headings

- **Purpose:** To structure content and create a logical hierarchy, making it easier for readers to scan and understand the topic.
- **Hierarchy:** Headings range from `<h1>` (the main title) to `<h6>` (the lowest level subheading).
- **Best Practices:** Use them frequently enough to help readers navigate, keep them brief, and maintain a clear, consistent hierarchy.

Paragraphs

- **Purpose:** To group sentences into logical blocks of text.
- **Functionality:** In HTML, a `<p>` tag is a container element that signals the start of a new paragraph. When a paragraph ends, the browser automatically adds a line break with extra space before the next one starts.
- **Usage:** Wrap your text within opening (`<p>`) and closing (`</p>`) tags.

Breaks

- **Purpose:** To force a new line without adding the extra vertical space that a paragraph tag provides.
- **Functionality:** The `<br>` tag is a single-line break element.
- **Usage:** It's useful for formatting text where you need a new line but don't want a new paragraph, such as in poetry or an address.

HTML Paragraphs

A paragraph always starts on a new line, and is usually a block of text. The HTML `<p>` element defines a paragraph. A paragraph always starts on a new line, and browsers automatically add some white space (a margin) before and after a paragraph.

Example

```
<p>This is a paragraph.</p>
```

HTML Display

You cannot be sure how HTML will be displayed. Large or small screens, and resized windows will create different results. With HTML, you cannot change the display by adding extra spaces or extra lines in your HTML code. The browser will automatically remove any extra spaces and lines when the page is displayed:

Example

```
<p>
```

```
This paragraph  
contains a lot of lines  
in the source code,  
but the browser  
ignores it.
```

```
</p>
```

```
<p>
```

```
This paragraph  
contains      a lot of spaces  
in the source    code,  
but the    browser  
ignores it.
```

```
</p>
```

4.4 Formatting Text

HTML offers various elements for formatting text, enhancing its visual presentation and sometimes conveying semantic meaning.

Common Text Formatting Elements:

- **Bold/Strong:**
 - `<b>`: Renders text in bold for visual emphasis.
 - `<strong>`: Renders text in bold and indicates strong importance or emphasis for screen readers and search engines.
- **Italic/Emphasized:**
 - `<i>`: Renders text in italics for visual distinction.
 - `<em>`: Renders text in italics and indicates emphasis for screen readers and search engines.
- **Underlined/Inserted:**
 - `<u>`: Underlines text (primarily for visual styling, though often discouraged in modern web design in favor of CSS).
 - `<ins>`: Represents inserted text, typically displayed with an underline.
- **Strikethrough/Deleted:**
 - `<del>`: Represents deleted text, typically displayed with a strikethrough.
- **Highlighting:**
 - `<mark>`: Highlights text, often with a yellow background, to draw attention to it.
- **Superscript and Subscript:**
 - `<sup>`: Renders text as superscript (e.g., for footnotes or exponents).
 - `<sub>`: Renders text as subscript (e.g., for chemical formulas).
- **Code and Preformatted Text:**
 - `<code>`: Displays a fragment of computer code in a monospaced font.
 - `<pre>`: Displays preformatted text, preserving whitespace and line breaks, often used for displaying code blocks or ASCII art.
- **Smaller Text:**
 - `<small>`: Renders text smaller than the surrounding text.

Example Usage:

Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Text Formatting Example</title>
</head>
<body>
  <p>This is <b>bold</b> text using the b tag.</p>
  <p>This is <strong>strong</strong> text using the strong tag.</p>
  <p>This is <i>italic</i> text using the i tag.</p>
  <p>This is <em>emphasized</em> text using the em tag.</p>
  <p>This is <del>deleted</del> text.</p>
  <p>This is <ins>inserted</ins> text.</p>
  <p>This is <mark>highlighted</mark> text.</p>
  <p>H<sub>2</sub>O (Subscript example)</p>
  <p>E = mc<sup>2</sup> (Superscript example)</p>
  <p>Here is some <code>console.log("code");</code></p>
  <pre>
    This text is
    preformatted and
    maintains its
    original spacing.
  </pre>
  <p>This is <small>small</small> text.</p>
</body>
</html>
```

Fonts

The HTML **** tag was historically used to define the font size, color, and face (font family) of text content within an HTML document. It allowed for direct styling of text within the HTML structure.

Attributes of the **** tag (in HTML 4.01 and earlier):

- **size:** Specifies the font size, typically as a numeric value from 1 to 7 (where 3 is the default) or a relative value like +2 or -3.
- **color:** Specifies the font color, using a named color (e.g., "blue"), a hexadecimal value (e.g., #0000FF), or an RGB value (e.g., rgb(0,0,255)).
- **face:** Specifies the font family (e.g., "Arial", "Verdana"). Multiple font names could be provided as a comma-separated list for fallback.

Example of **** tag usage (in older HTML versions): Code

```
<font size="4" color="blue" face="Verdana">This text will be blue, size 4, and in Verdana font.</font>
```

Deprecation in HTML5:

The **** tag has been deprecated in HTML5 and is no longer supported for styling purposes. Modern web development practices strongly advocate for using Cascading Style Sheets (CSS) to control the presentation and styling of web content.

bgcolor

The **bgcolor** attribute in HTML was traditionally used to set the background color of various elements, such as **<body>**, **<table>**, **<td>**, and **<tr>**. It allowed specifying a background color using color names (e.g., "red"), hexadecimal color codes (e.g., "#FF0000"), or RGB values (e.g., "rgb(255,0,0)").

The HTML **<body>** background attribute is used to specify a background image or color for the entire web page's body. You can provide either a URL for an image or a color code to customize the background of the HTML body element.

Example of bgcolor in a <body> tag:

Code

```
<!DOCTYPE html>
<html>
<body bgcolor="lightblue">
  <h1>Welcome to my webpage!</h1>
  <p>This page has a light blue background.</p>
</body>
</html>
```

All color values are created within an HTML tag by a six-digit hexadecimal number preceded by a hash (#) symbol. For example: “#FF0000” is the color red.

Color Name	HEX	Color
<u>Pink</u>	<u>#FFC0CB</u>	
<u>LightPink</u>	<u>#FFB6C1</u>	
<u>HotPink</u>	<u>#FF69B4</u>	
<u>DeepPink</u>	<u>#FF1493</u>	
<u>PaleVioletRed</u>	<u>#DB7093</u>	
<u>MediumVioletRed</u>	<u>#C71585</u>	

Special character and Symbol

Special characters in HTML, also known as HTML entities, are used to display characters that are either reserved in HTML (like < or >) or are not easily typed on a standard keyboard (like the copyright symbol or various mathematical symbols).

There are two primary ways to represent special characters in HTML:

- **Named Entities:** These use a name for the character, starting with an ampersand (&) and ending with a semicolon (;).

- Example: < for the less-than sign (<), > for the greater-than sign (>), & for the ampersand (&), " for the double quote ("), © for the copyright symbol (©).
- **Numeric Entities:** These use a numeric code for the character, either decimal or hexadecimal, starting with an ampersand (&) and a hash (#), and ending with a semicolon (;).
 - **Decimal:** &# followed by the decimal code.
 - Example: < for the less-than sign (<), > for the greater-than sign (>).
 - **Hexadecimal:** &#x followed by the hexadecimal code.
 - Example: & for the ampersand (&).

Why use special characters?

- **Reserved Characters:** To display characters like < and > directly in the text without the browser interpreting them as HTML tags.
- **Accessibility:** To ensure consistent display of characters across different browsers and operating systems.
- **Special Symbols:** To include symbols not readily available on a standard keyboard, such as currency symbols, mathematical operators, or decorative elements.
- **Invisible Characters:** To include characters like non-breaking spaces (or) for formatting and layout control.

Examples of common special characters:

- <: < or <
- >: > or >
- &: & or &
- ": " or "
- ': ' or '
- ©: © or ©
- ®: ® or ®
- €: € or €
- ' ' (**non-breaking space**): or

4.5 Working With List

The three list types. There are three list types in HTML: **unordered list**:- used to group a set of related items in no particular order. **ordered list**:- used to group a set of related items in a specific order. **description list** :- used to display name/value pairs such as terms and definitions

HTML provides three primary types of lists for structuring content:

1. Unordered Lists (): These lists display items without a specific order, typically marked with bullet points. Each item within an unordered list is defined by the (list item) tag.

Code:-

```
<ul>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ul>
```

2. Ordered Lists (): These lists display items in a specific sequence, typically marked with numbers or letters. Each item within an ordered list is also defined by the tag. The type attribute can be used to customize the numbering style (e.g., type="a" for lowercase letters, type="I" for uppercase Roman numerals).

<ol type = "i" start=START="3" > specify a starting number for an ordered list.

Type	Numbering Styles	
1	Arabic numbers	1,2,3,
a	Lower alpha	a, b, c,
A	Upper alpha	A, B, C,
i	Lower roman	i, ii, iii,
I	Upper roman	I, II, III,

Code:-

```
<ol>
  <li>First ordered item</li>
  <li>Second ordered item</li>
  <li>Third ordered item</li>
</ol>
```

3. Description Lists (<dl>): These lists are used to display name/value pairs, similar to a dictionary or glossary. They consist of a term (<dt>) and its corresponding description (<dd>).

The <dl>, <dt>, and <dd> tags in HTML are used together to create a description list (also sometimes called a definition list).

- **<dl> (Description List):** This element serves as the container for the entire description list. It encloses all the terms and their corresponding descriptions.
- **<dt> (Description Term):** This element defines a term or name within the description list. It is typically followed by one or more <dd> elements that provide the description for that term.
- **<dd> (Description Details):** This element provides the description, definition, or value for the preceding <dt> term. It contains the detailed explanation or information associated with the term

Example:

Code

```
<dl>
  <dt>Coffee</dt>
  <dd>A black hot drink made from roasted coffee beans.</dd>
  <dt>Milk</dt>
  <dd>A white liquid produced by mammals, often consumed as a beverage.</dd>
</dl>
```

In this example, "Coffee" and "Milk" are the terms defined by <dt>, and the sentences following them are their respective descriptions, defined by <dd>. The entire structure is contained within the <dl> element. This structure provides semantic meaning, indicating a relationship between terms and their descriptions, and can be styled using CSS for visual presentation.

Nested List

A nested ordered list in HTML is created by placing an (ordered list) element inside an (list item) element of another ordered list. This structure establishes a hierarchical relationship, where the inner list serves as a sub-list to a specific item of the outer list.

Here is an example of the HTML structure for a nested ordered list:

Code

```
<ol>
  <li>Main Item 1
    <ol>
      <li>Sub-item 1.1</li>
      <li>Sub-item 1.2</li>
    </ol>
  </li>
  <li>Main Item 2
    <ol>
      <li>Sub-item 2.1</li>
      <li>Sub-item 2.2
        <ol>
          <li>Sub-sub-item 2.2.1</li>
        </ol>
      </li>
    </ol>
  </li>
  <li>Main Item 3</li>
</ol>
```

Explanation:

- The outer tag defines the main ordered list.
- Each tag within the outer represents a main list item.
- To create a nested list, another tag is placed inside an tag of the parent list. This nested contains its own elements, which are the sub-items.
- This nesting can be done multiple times, creating deeper levels of hierarchy as shown with "Sub-sub-item 2.2.1".

Key Points:

- The nested list must be a child of an element. Placing it directly inside another but outside an will not create a proper nested structure.
- By default, ordered lists will use numbers for their markers. You can change the numbering type (e.g., Roman numerals, letters) using the type attribute on the tag (e.g., <ol type="a">).
- While HTML handles the basic nesting and numbering, more advanced hierarchical numbering schemes (like 1.1, 1.2) often require CSS counters.

4.6 Images and video

The HTML **** tag is used to embed an image in a web page.

Images are not technically inserted into a web page; images are linked to web pages. The **** tag creates a holding space for the referenced image.

The **** tag is empty, it contains attributes only, and does not have a closing tag.

The **** tag has two required attributes:

- **src** - Specifies the path to the image
- **alt** - Specifies an alternate text for the image

Syntax

```

```

The src Attribute

The required **src** attribute specifies the path (URL) to the image.

Note: When a web page loads, it is the browser, at that moment, that gets the image from a web server and inserts it into the page. Therefore, make sure that the image actually stays in the same spot in relation to the web page, otherwise your visitors will get a broken link icon. The broken link icon and the alt text are shown if the browser cannot find the image.

Example

```

```

The alt Attribute

The required **alt** attribute provides an alternate text for an image, if the user for some reason cannot view it (because of slow connection, an error in the src attribute, or if the user uses a screen reader).

The value of the alt attribute should describe the image:

Example

```

```

If a browser cannot find an image, it will display the value of the `alt` attribute:

Image Size - Width and Height

You can use the style attribute to specify the width and height of an image.

Example

```

```

Alternatively, you can use the width and height attributes:

Example

```

```

The width and height attributes always define the width and height of the image in pixels.

Note: Always specify the width and height of an image. If width and height are not specified, the web page might flicker while the image loads.

Video

The **<video>** tag is used to embed video content in a document, such as a movie clip or other video streams.

The **<video>** tag contains one or more **<source>** tags with different video sources. The browser will choose the first source it supports.

The text between the **<video>** and **</video>** tags will only be displayed in browsers that do not support the **<video>** element.

To show a video in HTML, use the **<video>** element:

Example

```
<video width="320" height="240" controls>  
  <source src="movie.mp4" type="video/mp4">  
  Your browser does not support the video tag.  
</video>
```

How it Works

The **controls** attribute adds video controls, like play, pause, and volume.

It is a good idea to always include width and height attributes. If height and width are not set, the page might flicker while the video loads.

The **<source>** element allows you to specify alternative video files which the browser may choose from. The browser will use the first recognized format.

The text between the **<video>** and **</video>** tags will only be displayed in browsers that do not support the **<video>** element.

HTML **<video>** Autoplay

To start a video automatically, use the **autoplay** attribute:

Example

```
<video width="320" height="240" autoplay>  
  <source src="movie.mp4" type="video/mp4">  
  Your browser does not support the video tag.  
</video>
```

Note: Chromium browsers do not allow autoplay in most cases. However, muted autoplay is always allowed.

Add **muted** after **autoplay** to let your video start playing automatically (but muted):

Example

```
<video width="320" height="240" autoplay muted>  
  <source src="movie.mp4" type="video/mp4">  
  Your browser does not support the video tag.  
</video>
```

Browser Support

The numbers in the table specify the first browser version that fully supports the **<video>** element.

Element					
<video>	4.0	9.0	3.5	4.0	10.5

4.7 Anchors, URLs and Image Maps

The Anchor element <a>:- The HTML <a> tag defines a hyperlink. It has the following syntax:

```
<a href="url">link text</a>
```

The most important attribute of the <a> element is the **href** attribute, which indicates the link's destination. The link text is the part that will be visible to the reader. Clicking on the link text, will send the reader to the specified URL address.

Example

This example shows how to create a link to google.com:

```
<a href="https://www.google.com/"> Google</a>
```

By default, links will appear as follows in all browsers:

- An unvisited link is underlined and blue
- A visited link is underlined and purple
- An active link is underlined and red

The <a> **HTML** element (or *anchor* element), with its **href** attribute, creates a hyperlink to web pages, files, email addresses, locations in the same page, or anything else a URL can address. Content within each <a> *should* indicate the link's destination. If the **href** attribute is present, pressing the enter key while focused on the <a> element will activate it.

Example

```
<html>
<body>
<ul>
  <li><a href="https://wu.edu.et">Wollo University</a></li>
  <li><a href="mailto:dbekele2003@gmail.com">Email</a></li>
  <li><a href="tel:+251911088630">Phone</a></li>
</ul>
</body>
</html>
```

A **mailto** link is a Uniform Resource Identifier (URI) scheme used in HTML to create hyperlinks that open the user's default email client with a new message pre-addressed to a specific email address. This link can also be customized to pre-fill the subject line, body content, and CC/BCC recipients. Clicking a mailto link automatically launches a new email, streamlining the process of sending an email directly from a website or document.

How it works

- When a user clicks a mailto link, their default email application (like Outlook, Gmail, or Apple Mail) automatically opens.
- The email window is pre-populated with the email address specified in the link, and any other parameters like the subject or body content.
- Users can then simply write and send their message without manually entering the recipient's address.

Customizing mailto links

- **Basic link:**
`<a href="mailto:recipient@example.com">Email Us</a>.`
- **Pre-fill subject:**
`<a href="mailto:recipient@example.com?subject=Hello%20there">Email Us</a>.` The %20 is a URL-encoded space.
- **Add CC/BCC:**
`<a href="mailto:recipit@example.com?cc=abc@example.com&bc=sec@example.com">Email Us</a>.`

Image Maps

The HTML **<map>** tag defines an image map. An image map is an image with clickable areas. The areas are defined with one or more **<area>** tags.

Try to click on the computer, phone, or the cup of coffee in the image below:



Example

Here is the HTML source code for the image map above:

```
<html>

<body>



  <map name="workmap">

    <area shape="rect" coords="34,44,270,350" alt="Computer" href="a.html">

    <area shape="rect" coords="290,172,333,250" alt="Phone" href="b.html">

    <area shape="circle" coords="337,300,44" alt="Coffee" href="c.html">

  </map>

</body>

</html>
```

How Does it Work?

The idea behind an image map is that you should be able to perform different actions depending on where in the image you click.

To create an image map you need an image, and some HTML code that describes the clickable areas.

The Image

The image is inserted using the `<img>` tag. The only difference from other images is that you must add a `usemap` attribute:

```

```

The `usemap` value starts with a hash tag `#` followed by the name of the image map, and is used to create a relationship between the image and the image map.

Create Image Map

Then, add a `<map>` element.

The `<map>` element is used to create an image map, and is linked to the image by using the required `name` attribute:

```
<map name="workmap">
```

The `name` attribute must have the same value as the `<img>`'s `usemap` attribute.

The Areas

Then, add the clickable areas.

A clickable area is defined using an `<area>` element.

Shape

You must define the shape of the clickable area, and you can choose one of these values:

- **rect** :- defines a rectangular region
- **circle** :- defines a circular region
- **poly** :- defines a polygonal region
- **default** :- defines the entire region

You must also define some coordinates to be able to place the clickable area onto the image.

Shape="rect"

The coordinates for `shape="rect"` come in pairs, one for the x-axis and one for the y-axis.

So, the coordinates **34,44** is located **34** pixels from the left margin and **44** pixels from the top:



The coordinates **270,350** is located **270** pixels from the left margin and **350** pixels from the top:



Now we have enough data to create a clickable rectangular area:

Example

```
<area shape="rect" coords="34, 44, 270, 350" href="a.html">
```

This is the area that becomes clickable and will send the user to the page "a.html"



Shape="circle"

To add a circle area, first locate the coordinates of the center of the circle: **337,300**



Then specify the radius of the circle: **44 pixels**



Now you have enough data to create a clickable circular area:

Example

```
<area shape="circle" coords="337, 300, 44" href="b.html">
```

This is the area that becomes clickable and will send the user to the page "b.html":



Shape="poly"

The shape="poly" contains several coordinate points, which creates a shape formed with straight lines (a polygon). This can be used to create any shape. Like maybe a croissant shape!

How can we make the croissant in the image below become a clickable link?



We have to find the x and y coordinates for all edges of the croissant:



The coordinates come in pairs, one for the x-axis and one for the y-axis:

Example

```
<area shape="poly"
```

```
coords="140,121,181,116,204,160,204,222,191,270,140,329,85,355,58,352,37,322,40,259,103,161,128,147" href="croissant.html">
```

This is the area that becomes clickable and will send the user to the page "croissant.html":



4.8 Tables

The **<table>** tag in HTML is used to create a table on a web page, organizing data into rows and columns. It serves as the main container for all table-related elements. HTML tables are used to display data in a structured format of rows and columns. The fundamental elements for creating an HTML table are:

- **<table>**: This element defines the entire table.
- **<tr>**: This element defines a table row, representing a horizontal line of data or headers within the table.
- **<th>**: This element defines a table header cell. By default, the text within <th> elements is bold and centered.
- **<td>**: This element defines a standard table data cell, containing the actual data for the table. By default, the text within <td> elements is regular and left-aligned.

Basic Table Structure Example:

Code:-

```
<table>
<tr>
  <th>Header 1</th>
  <th>Header 2</th>
</tr>
<tr>
  <td>Data 1</td>
  <td>Data 2</td>
</tr>
<tr>
  <td>Data 3</td>
  <td>Data 4</td>
</tr>
</table>
```

Here are the essential tags used within an HTML table:

- **<table>**: Defines the entire table structure.
- **<caption>**: Provides a caption or title for the table, typically displayed above it.
- **<colgroup>**: Specifies a group of one or more columns for formatting, often containing <col> elements.
- **<col>**: Defines properties for a single column within a <colgroup>.

- `<thead>`: Groups the header content of a table.
- `<tbody>`: Groups the body content of a table.
- `<tfoot>`: Groups the footer content of a table.
- `<tr>`: Defines a table row, which contains individual cells.
- `<th>`: Defines a header cell within a row, typically used for column or row titles.
- `<td>`: Defines a standard data cell within a row, holding the actual content.

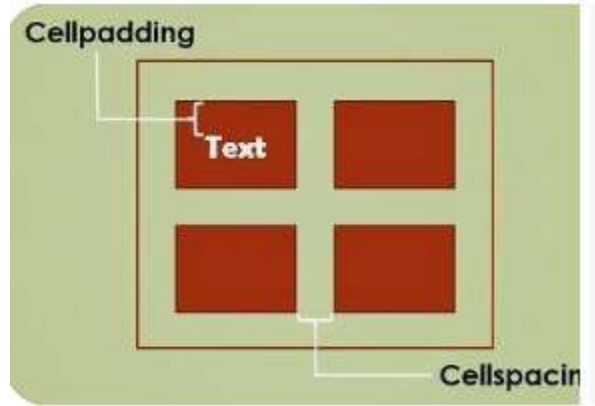
Example:

Code

```
<table>
  <caption>Monthly Savings</caption>
  <thead>
    <tr>
      <th>Month</th>
      <th>Savings</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>January</td>
      <td>10 000</td>
    </tr>
    <tr>
      <td>February</td>
      <td>15 000</td>
    </tr>
  </tbody>
</table>
```

Cellpadding and Cellspacing

Cellpadding and cellspacing are attributes used in HTML tables to control the spacing and layout of cells. While historically set directly in the `<table>` tag, modern web development practices favor using CSS for styling.



Cellpadding refers to the space between the content inside a cell and its border. It acts like an internal margin for each cell. Increasing cellpadding adds more space around the content within a cell, pushing it further away from the cell's edges.

Cellspacing refers to the space between individual table cells. It defines the gap between the borders of adjacent cells. Increasing cellspacing creates more visual separation between cells, making the table appear less dense.

Example:

```
<table border="1" cellpadding="10" cellspacing="15">
  <tr>
    <th>Header 1</th>
    <th>Header 2</th>
  </tr>
  <tr>
    <td>Data 1A</td>
    <td>Data 1B</td>
  </tr>
  <tr>
    <td>Data 2A</td>
    <td>Data 2B</td>
  </tr>
</table>
```

Colspan and Rowspan

In HTML tables, **colspan** and **rowspan** are attributes used within <th> (table header) or <td> (table data) elements to control how many columns or rows a single cell should span. This allows for the creation of more complex and visually organized table layouts by merging cells.

colspan attribute:

- The colspan attribute specifies the number of columns a cell should span horizontally.
- It is used when a single cell needs to extend across the width of multiple columns in a table row.
- The value of colspan is an integer representing the number of columns the cell will occupy.

Example of colspan:

Code

```
<table>
<tr>
  <th>Name</th>
  <th colspan="2">Contact Info</th>
</tr>
<tr>
  <td> Sara Mohammed</td>
  <td>sara@example.com</td>
  <td>+251911111111</td>
</tr>
</table>
```

In this example, the "Contact Info" header spans two columns, effectively merging the space that would otherwise be occupied by two separate header cells.

rowspan attribute:

- The rowspan attribute specifies the number of rows a cell should span vertically.
- It is used when a single cell needs to extend down across the height of multiple rows in a table column.
- The value of rowspan is an integer representing the number of rows the cell will occupy. A special value of 0 means the cell should span all remaining rows in the column.

Example of rowspan:

Code

```
<table>
  <tr>
    <th rowspan="2">Category</th>
    <th>Item 1</th>
  </tr>
  <tr>
    <th>Item 2</th>
  </tr>
  <tr>
    <td>Electronics</td>
    <td>Laptop</td>
  </tr>
  <tr>
    <td>Electronics</td>
    <td>Smartphone</td>
  </tr>
</table>
```

In this example, the "Category" header spans two rows, vertically merging the space for two rows of data.

Colspan and Rowspan Attributes

- You will use colspan attribute if you want to merge two or more columns into a single column.
- Similar way you will use rowspan if you want to merge two or more rows.

To draw this table:-

Column 1	Column 2	Column 3
Row 1 Cell 1	Row 1 Cell 2	Row 1 Cell 3
	Row 2 Cell 2	Row 2 Cell 3
Row 3 Cell 1		

```

<table border = "1">
<tr>
<th>Column 1</th>
<th>Column 2</th>
<th>Column 3</th>
</tr>
<tr>
<td rowspan = "2">Row 1 Cell 1</td>
<td>Row 1 Cell 2</td>
<td>Row 1 Cell 3</td>
</tr>
<tr>
<td>Row 2 Cell 2</td>
<td>Row 2 Cell 3</td>
</tr>
<tr>
<td colspan = "3">Row 3 Cell 1</td>
</tr>
</table>

```

4.9 HTML Frames

HTML frames were a feature in earlier versions of HTML that allowed web developers to divide a browser window into multiple, independent sections, each capable of displaying a separate HTML document. These sections were defined using the `<frameset>` and `<frame>` tags.

Key Components:

- **<frameset>**: This tag replaced the `<body>` tag in an HTML document and was used to define the layout of the frames within the browser window. It could specify whether the frames were arranged in rows or columns and their respective sizes.
- **<frame>**: This tag was placed within the `<frameset>` and specified the content (an HTML document) to be loaded into each individual frame. It included attributes like `src` (to link to the HTML file) and `name` (to identify the frame).
- **<noframes>**: This tag was used to provide alternative content for browsers that did not support frames.

How they worked:

A main HTML document would contain a `<frameset>` that defined the structure of the frames. Each `<frame>` within the `<frameset>` would then point to a separate HTML file, effectively embedding multiple independent web pages within a single browser window. This allowed for scenarios like a static navigation menu in one frame and dynamic content in another.

Current Status:

The use of `<frameset>` and `<frame>` tags is deprecated in HTML5 and is no longer recommended due to various drawbacks, including:

- **Accessibility issues:** Screen readers and other assistive technologies often struggled with frames.
- **Usability problems:** Bookmarking specific sub-pages within a frameset was difficult, and the browser's back button might not function as expected.
- **Search engine optimization (SEO) challenges:** Search engine crawlers had difficulty indexing content within frames.
- **Layout complexities:** Positioning frames with CSS was limited.

Modern Alternative:

The `<iframe>` tag is the modern and recommended way to embed content from another HTML document within a web page. Unlike `<frameset>`, `<iframe>` is used within the `<body>` of an HTML document and embeds a single, independent frame, providing more flexibility and addressing many of the issues associated with traditional frames.

The **`<iframe>`** tag is used in HTML to embed another HTML document within the current one. While the **`<frame>`** and **`<frameset>`** tags were used in older HTML versions to divide a browser window into multiple frames, they are now considered deprecated and are not recommended for modern web development. The `<iframe>` tag is the current and preferred method for embedding external content.

Here is an example of an <iframe> tag:

Code

```
<!DOCTYPE html>
<html>
<head>
  <title>iframe Example</title>
</head>
<body>
  <h1>My Main Page Content</h1>
  <p>This is some content on the main page.</p>
  <iframe src="xxx.html" width="600" height="400" title="Embedded Content"></iframe>
  <p>More content after the embedded frame.</p>
</body>
</html>
```

In this example:

- src="xxx.html" specifies the URL of the HTML document to be embedded. This can be a relative path to a local file or an absolute URL to an external website.
- width="600" and height="400" define the dimensions of the iframe in pixels.
- title="Embedded Content" provides an accessible title for the iframe, which is important for screen readers and accessibility.

The <iframe> tag allows for embedding various types of content, such as other HTML pages, videos, or interactive elements, within a designated area of a webpage

4.10 HTML Forms

An HTML form is used to collect user input. The user input is most often sent to a server for processing.

Example

First name:

Last name:

HTML Forms are required, when you want to collect some data from the site visitor.

- For example, during user registration you would like to collect information such as name, email address, credit card, etc.

A form will take input from the site visitor and then will post it to a back-end application. The back-end application will perform required processing on the passed data based on defined business logic inside the application. There are various form elements available like text fields, textarea fields, drop-down menus, radio buttons, checkboxes, etc.

The HTML <form> tag is used to create an HTML form and it has following syntax :-

```
<form action = "Script URL" method = "GET|POST">
```

form elements like input, textarea etc.

```
</form>
```

Form Attributes

- **action**
 - Backend script ready to process your passed data.
 - Specifies an address (url) where to submit the form (default: the submitting page).
- **method**
 - Method to be used to upload data. The most frequently used are GET and POST methods.
 - Specifies HTTP method used when submitting form (default: GET).
- **target**
 - Specify the target window or frame where the result of the script will be displayed. It takes values like _blank, _self, _parent etc.
- **enctype**
 - You can use the enctype attribute to specify how the browser encodes the data before it sends it to the server. It is only applicable if we use the POST method.
 - Possible values are –
 - application/x-www-form-urlencoded – This is the standard method most forms use in simple scenarios.
 - multipart/form-data – This is used when you want to upload binary data in

the form of files like image, word file etc.

- **name:** Specifies a name used to identify the form
- **novalidate:** specifies that the browser should not validate the form

HTML Form Controls

There are different types of form controls that you can use to collect data using HTML form

1. Text Input Controls
2. Checkboxes Controls
3. Radio Box Controls
4. Select Box Controls
5. File Upload Box
6. Button Control

The <input> Element

The HTML <input> element is the most used form element. An <input> element can be displayed in many ways, depending on the type attribute.

Here are some examples:

Type	Description
<input type="text">	Displays a single-line text input field
<input type="radio">	Displays a radio button (for selecting one of many choices)
<input type="checkbox">	Displays a checkbox (for selecting zero or more of many choices)
<input type="submit">	Displays a submit button (for submitting the form)
<input type="button">	Displays a clickable button

1. Text Input Controls

- Input tags are HTML elements designed to take a specific form of input from users
- List of attributes for <input> tag for creating text field
 - **type:-** Indicates the type of input control and for text input control it will be set to text.
 - **name :-** Used to give a name to the control which is sent to the server to be

recognized and get the value.

- **value:-**This can be used to provide an initial value inside the control.
- **size:-** Allows to specify the width of the text-input control in terms of characters.
- **maxlength:-** Allows to specify the maximum number of characters a user can enter into the text box.

There are three types of text input used on forms –

I. Single-line text input controls

- This control is used for items that require only one line of user input, such as search boxes or names.

They are created using HTML `<input>` tag.

```
<form >
    First name: <input type = "text" name = "first_name" />
    <br>
    Last name: <input type = "text" name = "last_name" />
</form>
```

It looks like in the browser

First name:

Last name:

II. Password input controls

- This is also a single-line text input but it masks the character as soon as a user enters it. They are also created using HTML `<input>` tag but type attribute is set to password.
- Password: `<input type = "password" name = "password" />`

III. Multiple-Line Text Input Controls

- This is used when the user is required to give details that may be longer than a single sentence. Multi-line input controls are created using HTML `<textarea>` tag.
- attributes for `<textarea>` tag.
 - **rows:** indicates the number of rows of text area box.
 - **cols:** indicates the number of columns of text area box

```

<form>
  Description : <br />
  <textarea rows = "5" cols = "50" name = "description">
    Enter description here...
  </textarea>
</form>

```

It looks like in the browser

Description :

Enter description here...

2. Checkbox Control

- used to allow a user to select one or more options from a limited number of options. They are also created using HTML <input> tag but type attribute is set to checkbox...
- attributes for <checkbox> tag.
 - **type**: Indicates the type of input control and for checkbox input control it will be set to checkbox..
 - **name**: Used to give a name to the control which is sent to the server to be recognized and get the value.
 - **value** :The value that will be used if the checkbox is selected.
 - **checked**: Set to checked if you want to select it by default.

```

<form>
  <input type = "checkbox" name = "maths" value = "on"> Maths <br>
  <input type = "checkbox" name = "physics" value = "on"> Physics
</form>

```

It looks like in the browser

☐ Maths

☐ Physics

3. Radio Button Control

- used when out of many options, just one option is required to be selected. Using HTML `<input>` tag but type attribute is set to radio.
- Attribute and Description
 - **type:** Indicates the type of input control and for checkbox input control it will be set to a radio.
 - **name:** Used to give a name to the control which is sent to the server to be recognized and get the value.
 - **value:** The value that will be used if the checkbox is selected
 - **checked:** Set to checked if you want to select it by default

`<form>`

`<input type="radio" name="subject" value="maths"> Maths <br>`

`<input type="radio" name="subject" value="physics"> Physics`

`</form>`

It looks like in the browser

☐ Maths
☐ Physics

4. Select Box Controls

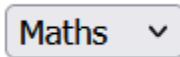
- also called drop down box provides option to list down various options in the form of drop down list, from where a user can select one or more options.
- Following is the list of important attributes of `<select>` tag and its description
 - **name:** Used to give a name to the control which is sent to the server to be recognized and get the value.
 - **size:** This can be used to present a scrolling list box.
 - **multiple:** If set to "multiple" then allows a user to select multiple items from the menu.
- Following is the list of important attributes of `<option>` tag and its description
 - **value:** The value that will be used if an option in the select box box is selected.
 - **selected:** Specifies that this option should be the initially selected value when the page loads.
 - **label:** An alternative way of labeling options

```

<form>
  <select name="dropdown">
    <option value="Maths" selected>Maths</option>
    <option value="Physics">Physics</option></select>
  </form>

```

It looks like in the browser



5. File Upload Box

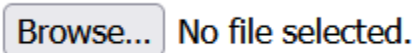
- If you want to allow a user to upload a file to your web site, you will need to use a file upload box, also known as a file select box.
- This is also created using the `<input>` element but type attribute is set to file.
- Following is the list of important attributes of file upload box and its description
 - **name:** Used to give a name to the control which is sent to the server to be recognized and get the
 - **value:** accept: Specifies the types of files that the server accepts.

```

<form>
  <input type = "file" name = "fileupload" accept = "image/*" />
</form>

```

It looks like in the browser



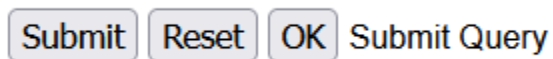
6. Button Controls

- You can also create a clickable button using `<input>` tag by setting its type attribute to button.
- Type of Button
 - **submit:** This creates a button that automatically submits a form.
 - **reset:** This creates a button that automatically resets form controls to their initial values.
 - **button:** This creates a button that is used to trigger a client-side script when the user clicks that button.

- **image:** This creates a clickable button but we can use an image as background of the button.
- Description of the attributes
 - **name:** The name attribute uniquely identifies a button
 - **value:** The value attribute gives a label to the button

```
<form>
  <input type = "submit" name = "submit" value = "Submit" />
  <input type = "reset" name = "reset" value = "Reset" />
  <input type = "button" name = "ok" value = "OK" />
  <input type = "image" name = "imagebutton" src = "xxx.jpg" />
</form>
```

It looks like in the browser



Hidden Controls

- used to hide data inside the page which later on can be pushed to the server. This control hides inside the code and does not appear on the actual page.

```
<form>
  <p>This is page 10</p>
  <input type = "hidden" name = "pagename" value = "10" />
  <input type = "submit" name = "submit" value = "Submit" />
  <input type = "reset" name = "reset" value = "Reset" />
</form>
```

It looks like in the browser

This is page 10



The Submit and Reset Button

- Defines a button for submitting a form to a form-handler
 <input type= "reset"> – clears all data entered so far

HTML Form Handlers

- When a user clicks the submit button, the form needs to send that information somewhere to be processed.
- This is where the form handler comes in.
 - The form-handler is specified in the form's action attribute:
 - To be submitted correctly, each input field must have name attribute.

```
<form action="action_page.php">
```

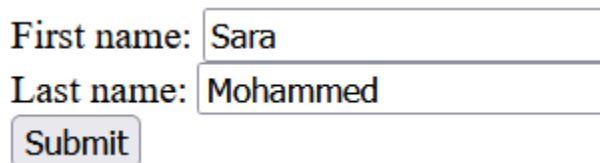
```
  First name: <input type="text" name="firstname" value="Sara"><br>
```

```
  Last name: <input type="text" name="lastname" value="Mohammed"><br>
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

It looks like in the browser



First name:

Last name:

Action attribute: - defines the action to be performed when the form is submitted

- The common way to submit a form to a server, is by using a submit button.

```
<form action="action_page.php">
```
- In the example above, server-side script (PHP program) is specified to handle the submitted form (process the form data):
- If action attribute is omitted, the action is set to current page

HTTP works as a request-response protocol between a client and server. The HTTP protocol allows a variety of mechanisms for sending requests to a server. The most commonly used HTTP request methods are

GET:

- This method is used to retrieve data from a resource.
- It's the most common method used when browsing websites, as it tells the server to send back the information for the requested resource, like a web page or an image.

- Less secure because it exposes data in the URL .
- Can only send a limited amount of data
- It is good for search content in order to let users to used it again.

<form action="action_page.php" method="get">

POST:

- This method is typically used to submit data to a server.
- For instance, when you fill out a form on a website and click submit, the browser uses a POST request to send the form data to the server for processing (like creating an account or placing an order).
- Offers greater security

<form action="action_page.php" method="post">

- Transmits data within the HTTP request body
- Can handle larger amounts of data
- Better suited for sending sensitive information

PUT: This method is used to update or completely replace a resource with the data sent in the request.

- It's often used with RESTful APIs for specific data manipulation

4.11 Special HTML Elements

Special HTML elements" can refer to a few distinct categories:

1. HTML Entities (Special Characters):

These are characters that cannot be directly typed or are reserved in HTML, such as the less-than sign (<), greater-than sign (>), or the ampersand (&). They are represented using named entities (e.g., <, >, &) or numeric entities (e.g., <, >, &). Other examples include:

- Copyright symbol: © or ©
- Registered trademark symbol: ® or ®
- Non-breaking space: or

2. Semantic HTML Elements:

These elements provide meaning to the content they enclose and improving accessibility. They describe the purpose of the content rather than just its presentation. Examples include:

- `<header>`: Defines the introductory content or a set of navigational links.
- `<nav>`: Defines a set of navigation links.
- `<main>`: Defines the dominant content of the `<body>`.
- `<article>`: Represents self-contained content, like a blog post or news article.
- `<section>`: Defines a section in a document.
- `<footer>`: Defines the footer for a document or section.
- `<aside>`: Defines content aside from the content it is placed in (like a sidebar).
- `<figure>` and `<figcaption>`: Used for self-contained content like images, diagrams, or code listings with a caption.

3. HTML Formatting Elements:

These elements are used to display special types of text, affecting its visual presentation.

- `<b>`: Bold text
- `<strong>`: Important text (semantically strong emphasis)
- `<i>`: Italic text
- `<em>`: Emphasized text (semantically emphasized)
- `<mark>`: Marked or highlighted text
- `<small>`: Smaller text
- `<del>`: Deleted (strike-through) text
- `<ins>`: Inserted (underlined) text
- `<sub>`: Subscript text
- `<sup>`: Superscript text

4. Other Specialized Elements:

Certain elements serve specific functions beyond basic structure or text formatting.

- `<details>` and `<summary>`: Create an expandable/collapsible widget.
- `<time>`: Represents a specific period in time.
- `<datalist>`: Provides a list of pre-defined options for an `<input>` element.
- `<audio>` `

4.12 Page Layout and Design Considerations

Key page layout and design considerations for a website include focusing on user needs and mobile responsiveness, establishing a clear visual hierarchy, and ensuring a consistent and scannable design. Important elements include a clear header, organized content with headings and whitespace, legible typography, and prominent calls-to-action. It is also critical to optimize for performance, test the site with real users, and ensure the design supports the site's overall purpose and goals.

User experience and accessibility

- **Prioritize users:** Design with your target audience's needs and habits in mind to make their experience intuitive.
- **Ensure responsiveness:** Use a responsive design so the site adapts smoothly to all screen sizes, from desktops to mobile phones.
- **Focus on legibility:** Use high-contrast colors for text and backgrounds, use clear fonts, and avoid excessive use of all-caps or other distracting formatting.
- **Optimize for performance:** Keep images and other media optimized to minimize download times.
- **Avoid intrusive elements:** Do not use features like auto-playing media that can be disruptive.

Layout and structure

- **Establish a clear hierarchy:** Use headings, whitespace, and visual contrast to guide the user's eye and indicate the importance of different elements.
- **Use whitespace effectively:** Strategic use of whitespace prevents clutter and improves readability and navigation.
- **Organize content logically:** Group similar or related elements together and use a consistent layout structure, such as rows, to present ideas visually.
- **Provide clear navigation:** Ensure users can easily find their way around the site with a clear header and calls-to-action.
- **Minimize horizontal scrolling:** Avoid layouts that force users to scroll side-to-side.

Visual consistency

- **Maintain consistency:** Use a consistent style for colors, typography, images, and graphic elements across all pages to create a cohesive brand identity.
- **Use color thoughtfully:** Use complementary colors that are easy on the eyes and provide enough contrast for text.
- **Choose images wisely:** Use images that support the content without overwhelming it. Resize images to fit the layout and consider using smaller thumbnails that link to larger versions.

Strategy and goals

- **Define your purpose:** Your design should be built around the website's goals, whether it's to sell products, provide information, or capture leads.
- **Test and iterate:** Gather user feedback and conduct testing to identify and fix usability issues.

Example:-

HTML layout example is achieved using a table with 3 rows and 2 columns but the header and footer column spans both columns using the colspan attribute –

Nested Tables

- You can use one table inside another table. Not only tables you can use almost all the tags inside table data tag <td>.

```
<!DOCTYPE html>
<html>
<head>
<title>HTML Layout using Tables</title>
</head>
<body>
<table width = "100%" border = "0">
<tr>
<td colspan = "2" bgcolor = "#b5dcb3">
<h1>This is Web Page Main title</h1>
</td>
</tr>
<tr valign = "top">
```

```

<td bgcolor = "#aaa" width = "50">
<b>Main Menu</b><br />
HTML<br />
PHP<br />
PERL...
</td>
<td bgcolor = "#eee" width = "100" height = "200">
Technical and Managerial Tutorials
</td>
</tr>
<tr>
<td colspan = "2" bgcolor = "#b5dcb3">
<center>
Copyright © 2007 Tutorialspoint.com
</center>
</td>
</tr>
</table>
</body>
</html>

```

